Toward Reliable Detection of Android Lifecycle Violations with Static Analysis and Domain-Adapted Code Models

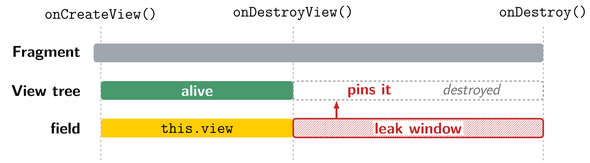
Muhammad Tehmasib Ali Tashfeen Advisor: Dr. Zhiyong Shan

School of Computing, Wichita State University

Thesis

Reliable lifecycle detection needs **traceable static evidence** *and* **domain-adapted contextual reasoning**.

The Bug: A Reference Outlives Its Owner




```
public class MusicianFragment extends Fragment {
  private View view;                                  retain

  public View onCreateView(LayoutInflater inf, ...) {
    view = inf.inflate(...);
    return view;
  }
  // no onDestroyView()                               no cleanup
}
```

Pattern from a real LeakScope finding (simplified).

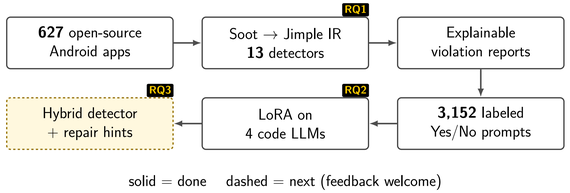
Three Research Questions

RQ1 Turn Android docs into **explainable static rules**

RQ2 Does **LoRA** make local code models useful?

RQ3 **Fuse** rule evidence + model judgement

Dissertation Pipeline



solid = done dashed = next (feedback welcome)

Feedback I'm Seeking

- Fusion policy when rule and model **disagree**?
- **Kotlin**: flows, lambdas, lifecycle-aware APIs
- How to evaluate **repair hints** with developers?

RQ1 | Every Rule Is a Three-Predicate Cascade

$$\underbrace{P_{\text{component}}}_{\text{is it a Fragment?}} \wedge \underbrace{P_{\text{retention}}}_{\text{field} \leftarrow \text{inflated View}} \wedge \neg \underbrace{P_{\text{cleanup}}}_{\text{nulled in onDestroyView?}}$$

Core of `FragmentViewFieldRetentionLeakDetector` (Soot / Jimple)

```
if (!isFragment(cls)) return "";                                  P_comp

// pass 1: locals produced by inflate / findViewById / bind
if (inViewLifecycle && isViewCreation(rhs)) viewLocals.add(lhs);

// pass 2: this.field = <view local>                              P_ret
if (isViewOrBindingType(field.getType()) && isViewValue)
    retained.put(field, methodName);

// cleared?  this.field = null  inside onDestroyView              ¬P_clean
if (isDestroyView && rhs instanceof NullConstant) cleared.add(field);

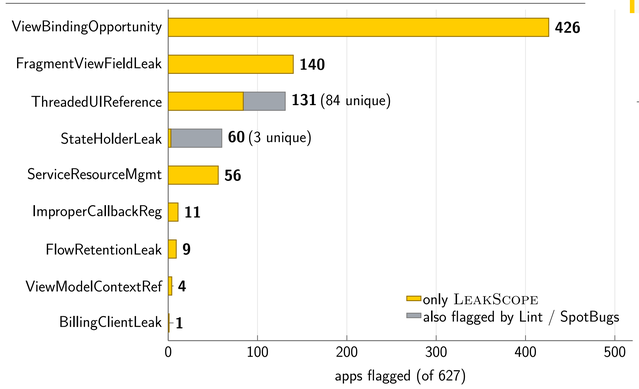
retained.keySet().removeAll(cleared);  // remainder => report
```

Real report emitted by LeakScope

```
Fragment View Field Retention Leak Detected
Class:   com.example.quanlyamnhac.fragment.MusicianFragment
Issue:   onDestroyView() is missing
Leaked: view : android.view.View  (assigned in onCreateView)
Fix:     override onDestroyView() { ...; view = null; }
```

Every warning names *what* is retained, *where* it was assigned, and the *missing* cleanup.

Most Findings Are Invisible to Lint & SpotBugs



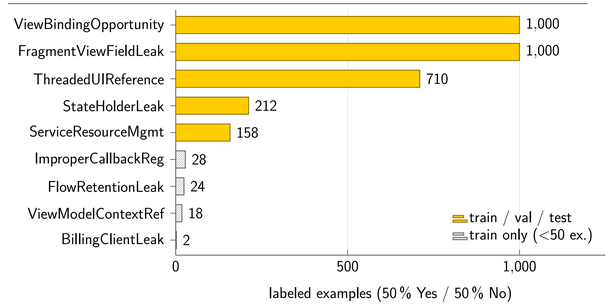
734
app–category pairs flagged *only* by LeakScope

485
of 627 apps flagged by LeakScope
SpotBugs 344 · Lint 106

Where Lint Still Wins (Honest Gaps)

- `static Context` in **Application** subclasses — our rule scans Activity/Fragment only
- **Anonymous `Handler`** capturing its outer Activity
- ⇒ StateHolderLeak: Lint 70 apps vs. LeakScope 60; extending scope is planned

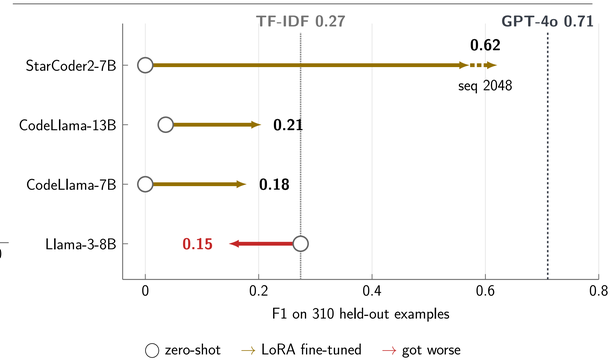
RQ2 | Detector Output Becomes Training Data




```
### Instruction: Does this Android code contain a
  ThreadedUIReference violation (worker thread captures UI)?
### Input:       <Java source, ≤ 2,500 chars>
### Response:    Yes | No  + detector evidence
  Captured UI objects: r0 : ...StackFragment
```

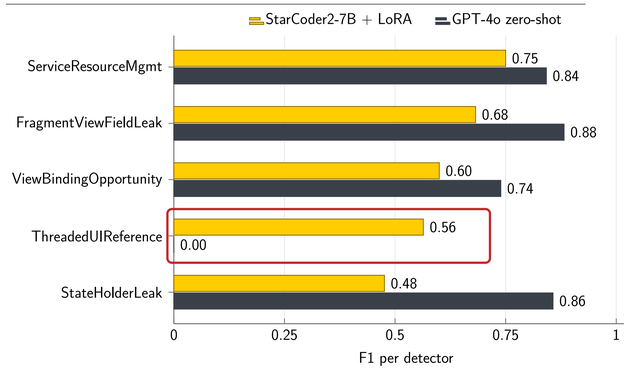
3,152 prompts · 2,535 / 307 / 310 split · stratified per detector

LoRA Turns StarCoder2 From F1 0.00 Into 0.62



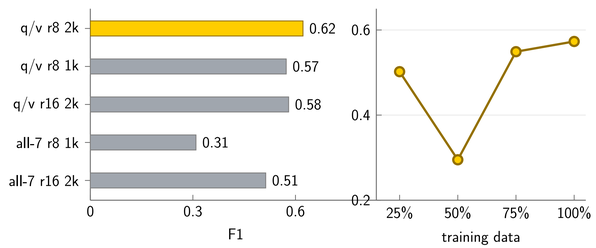
- Best local: **recall 0.76** but **precision 0.53** — over-flags
- Only StarCoder2 learns the task; others stay < 0.21

GPT-4o Misses All 37 ThreadedUIReference Bugs



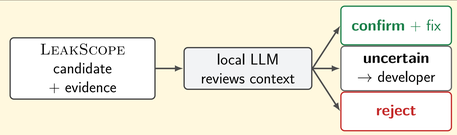
GPT-4o: *"the snippet primarily consists of import statements…"* — truncated inputs hide the capture site; the fine-tuned model still finds 22 of 37.

Longer Context Helps; Wider LoRA Hurts



StarCoder2-7B. Data scaling is **non-monotonic** (50 % dips to 0.30) — rare families need better sampling.

RQ3 | Next: Hybrid Detector



Rules supply **traceable evidence**; the model supplies **context** to cut false alarms.

Takeaways

1. **Rules find what others miss** — 734 unique app–category hits
2. **LoRA is necessary, not sufficient** — 0.00 → 0.62, GPT-4o 0.71
3. **Each fails differently** ⇒ fuse them